COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	10 - 15 - 2026	**DATE SUBMITTED:**	10 - 19 - 2026
NAME:	Canoy Hail B. Mauro, Ryan Lewis B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 13 **Parallel Algorithms and Multithreading**

1. PROCEDURE OUTPUT

Screenshot

```cpp
#include <iostream>
#include <string>
#include <thread>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");

    t1.join();

    return 0;
}
```

Output 1:

```
Printing integer: 10
Printing string: T.I.P.
```

Output 2:

```
Printing integer: 10
Printing string: T.I.P.
PS C:\Users\TIPQC\Downloads\hoa13>
```

Analysis

The program creates a thread named t1 that executes the print() function. It displays the integer value 10 and the string "T.I.P.". The join() function makes the main thread wait until t1 finishes its task. This example demonstrates how to create and execute a thread in C++.

Table 13-1. Simple One-Threaded Example

Screenshot

```cpp
#include <iostream>
#include <string>
#include <thread>
#include <vector>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };

    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }

    return 0;
}
```

Output 1:

```
2 : Computer0 : T.I.P.

1 : Competent
3 : Engineers
PS C:\Users\TIPQC\Docume
 .\CPE010_HOA13p1_Mauro_
0 : T.I.P.3 : Engineers
1 : Competent
2 : Computer

PS C:\Users\TIPQC\Docume
 .\CPE010_HOA13p1_Mauro_
1 : Competent
2 : Computer
3 : Engineers
0 : T.I.P.
```

Output 2:

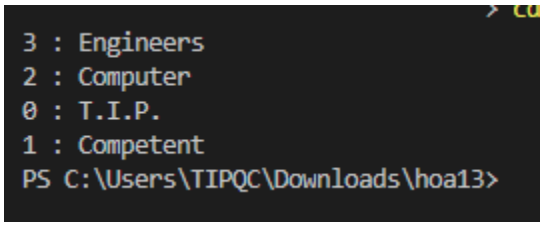
	
Analysis	The program demonstrates multithreading which allows the program to execute 2 or more tasks at the same time. The output order can be different each time the program runs because each task can benefit from the parallel execution.

Table 13-2. Multithreaded Example

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. Write a definition of multithreading and its advantages/disadvantages.	
Canoy Hail B.	Multithreading allows a program to perform different tasks using multiple threads, making it faster and more responsive, but it can also cause errors when threads access the same data.
Mauro, Ryan Lewis B.	Multithreading is a technique in programming where multiple threads work on different parts of a program, improving performance and responsiveness, although it can be harder to debug and manage.

2. Rationalize the use of multithreading by providing at least 3 use-cases.	
Canoy Hail B.	Multithreading is useful in online games, web servers that handle many users, and applications that download files while allowing users to do other tasks.
Mauro, Ryan Lewis B.	It is commonly used in video editing to process tasks, web browsers to keep pages responsive, and messaging apps to receive messages while the user types.

3. Differentiate between parallelism and concurrency.	
Canoy Hail B.	Concurrency means managing multiple tasks over overlapping periods, while parallelism means executing multiple tasks at exactly the same time.
Mauro, Ryan Lewis B.	Concurrency allows several tasks to make progress within the same period, even by switching between them. Parallelism happens when multiple tasks run simultaneously

	on different processing units.

Part B: Create C++ Code and show a solution that satisfies the given requirements below	
Code	

```cpp
#include <iostream>  // Used for displaying output
#include <thread>    // Used to create threads
#include <mutex>     // Used to protect shared data

using namespace std;

int total = 0;  // Global variable shared by all threads
mutex mtx;      // Prevents simultaneous updates to total

// Adds the given number to the global variable
void add(int number) {
    lock_guard<mutex> lock(mtx);
    total += number;
}

int main() {
    // Create three threads with different values
    thread T1(add, 10);
    thread T2(add, 50);
    thread T3(add, 90);

    // Wait for T1, T2, T3 to finish and display the total
    T1.join();
    cout << "After T1: " << total << endl;

    T2.join();
    cout << "After T2: " << total << endl;

    T3.join();
    cout << "After T3: " << total << endl;

    return 0;
}
```

Output

```
After T1: 150
After T2: 150
After T3: 150
```

Analysis	
Canoy Hail B.	The code uses three threads to add 10, 50, and 90 to the global variable total. The add() function performs the addition, while the mutex prevents threads from updating

	the variable at the same time. The join() function waits for each thread to finish before displaying the total.
Mauro, Ryan Lewis B.	This program demonstrates how multithreading works in C++ by allowing three threads to add values to one shared variable. Since the threads can finish in any order, the displayed totals may vary, but the final total will always be 150.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result

Code

```cpp
#include <iostream>
#include <thread>
using namespace std;

int fNum, lNum;
int result = 0;

// Compares the two numbers
void greaterNum() {
    if (fNum > lNum) {
        result = 1;
    }
    else if (lNum > fNum) {
        result = 2;
    }
    else {
        result = 3;
    }
}

int main() {
    cout << "What Number is Greater Calculator" << endl;

    cout << "Enter First Number: ";
    cin >> fNum;

    cout << "Enter Second Number: ";
    cin >> lNum;

    // Create a thread to compare the numbers
    thread T1(greaterNum);

    // Wait for the thread to finish
    T1.join();

    // Display the result
    if (result == 1) {
        cout << fNum << " is Greater than " << lNum << endl;
    }
    else if (result == 2) {
        cout << lNum << " is Greater than " << fNum << endl;
    }
    else {
        cout << fNum << " is Equal to " << lNum << endl;
    }

    return 0;
}
```

Output

```
PS C:\Users\Ian Canoy\Downloads\hoa13\output> cd 'c:\Users\Ian Canoy\Downloads\hoa13\output'
PS C:\Users\Ian Canoy\Downloads\hoa13\output> & .\'supp.exe'
What Number is Greater Calculator
Enter First Number: 80
Enter Second Number: 100
100 is Greater than 80
```

Analysis	
Canoy Hail B.	We used the Hands-on Activity 1.1: Basic C++ Programming supplementary activity as the basis for our Greater Number Calculator. We added multithreading so the comparison of the two numbers runs in a separate thread, and we used T1.join() to wait for the comparison to finish before showing the answer.
Mauro, Ryan Lewis B	We improved the original program by adding a thread to check which number is greater or if both numbers are equal. Through this activity, we learned how thread and join() work together to complete a task before displaying the result.

Conclusion (Canoy Hail B.)
I learned a lot from this activity, especially how multithreading works in C++ and how it can run a function in a separate thread. I also understood how join() helps the program wait for the thread to finish before displaying the result.
Conclusion (Mauro, Ryan Lewis B.)
Through this activity, I learned how to improve a basic C++ program by adding multithreading. I gained a better understanding of how threads work and how they can be used to perform tasks separately within a program.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **none**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **none**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **none**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.